

Отчёт по лабораторной работе №21

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-25 Богатырева Екатерина Денисовна № по списку 2.

Контакты: bogatyrevaK2019@yandex.ru

Работа выполнена: «12» марта 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «17» марта 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** Программирование на интерпретируемых командных языках ОС UNIX.
2. **Цель работы:** Разработать сценарий (скрипт) для командного интерпретатора Bash и программу на языке Python для объединения всех файлов с заданным суффиксом, размер которых не превышает указанного, в один результирующий файл.
3. **Задание (Вариант № 21):** Объединение в один файл всех файлов с указанным суффиксом, имеющих размер меньше заданного.
4. **Оборудование:** *Оборудование ПЭВМ студента, если использовалось:* Процессор: Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 ГБ, SSD 256 ГБ
5. **Программное обеспечение:** *Программное обеспечение ЭВМ студента, если использовалось:* Операционная система семейства Unix, наименование Ubuntu, версия 24.04 LTS, интерпретатор команд `bash` версия 5.2.21. Система программирования: gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) версия 11.4 Редактор текстов: VS code версия: 1.107.1. Язык программирования: bash, python.
6. **Идея, метод, алгоритм решения задачи [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:**
Идея: Необходимо рекурсивно обойти все файлы в указанном каталоге, отобрать те, у которых расширение совпадает с заданным, и чей размер меньше порогового значения, а затем последовательно записать их содержимое в один выходной файл.

Метод:

1. Получение аргументов: Скрипт принимает четыре параметра: суффикс (расширение), максимальный размер в байтах, каталог для поиска и имя выходного файла.
2. Рекурсивный обход: Используя `os.walk()` в Python и `find` в Bash, выполняется обход всех файлов в указанном каталоге и его поддиректориях.
3. Фильтрация по суффиксу и размеру: Для каждого найденного файла проверяется, оканчивается ли его имя на заданный суффикс и меньше ли его размер указанного значения.
4. Запись в выходной файл: Если оба условия выполнены, содержимое файла дописывается в выходной файл. Для каждого добавленного файла выводится диагностическое сообщение.

Алгоритм:

1. суффикс = аргумент1
2. макс_размер = аргумент2
3. каталог = аргумент3
4. выходной_файл = аргумент4
5. очистить(выходной_файл)
6. Для каждого файла в рекурсивный_обход(каталог):

7. если файл оканчивается на "." + суффикс:
8. если размер(файл) < макс_размер:
9. напечатать("adding", файл)
10. дописать_содержимое(файл) в выходной_файл

7. Сценарий выполнения работы (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию). **План работы:**

1. Анализ предметной области: изучить способы рекурсивного обхода файловой системы в Bash и Python.
2. Проектирование алгоритма: определить условия отбора файлов (суффикс, размер).
3. Реализация на Bash:
 - Использовать find для обхода.
 - Использовать stat -c%s для получения размера.
 - Использовать перенаправление вывода для записи в файл.
4. Реализация на Python:
 - Использовать модули os и argparse.
 - Применить os.walk() для обхода.
 - Использовать os.path.getsize() для получения размера.
5. Тестирование: проверка на различных структурах каталогов и файлов.

Тесты:

```
mkdir test_merge
cd test_merge
```

```
echo hello > a.txt
echo world > b.txt
echo veryverylongtext > c.txt
```

```
mkdir subdir
echo sub > subdir/d.txt
echo longlonglongtext > subdir/e.txt
```

```
ls
ls subdir
```

```
python3 1.py txt 10 . result.txt # или bash 1.sh txt 10 . result.txt
```

```
ls
ls subdir
cat result.txt
```

Соображения по тестированию:

- Проверка на пустом каталоге: скрипт должен создавать пустой выходной файл.
- Проверка с файлами разного размера: в выходной файл попадают только файлы, размер которых меньше указанного.
- Проверка с файлами в поддиректориях: убедиться, что обход работает рекурсивно.
- Проверка с разными суффиксами: файлы с другим расширением игнорируются.

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. **Подпись преподавателя** _____

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#!/bin/bash
```

```

help() {
    echo "Usage: $0 [OPTION]... SUFFIX SIZE DIR OUTFILE"

    echo

    echo "Merge into OUTFILE all files with SUFFIX whose size is less than SIZE bytes inside DIR and its
subdirectories."

    echo

    echo "Options:"

    echo " -h, --help    show this help message and exit"

    echo

    echo "Exit status:"

    echo " 0   if OK,"

    echo " 1   if invalid arguments or missing parameters"
}

if [[ "$1" == "-h" || "$1" == "--help" ]]; then
    help
    exit 0
fi

if [ $# -ne 4 ]; then
    help
    exit 1
fi

suffix=$1
size=$2
dir=$3
outfile=$4
> "$outfile"

for f in $(find "$dir" -type f -name ".*$suffix"); do
    filesize=$(stat -c%s "$f")
    if [ "$filesize" -lt "$size" ]; then
        echo "adding $f"
        cat "$f" >> "$outfile"
    fi
done

///

#!/usr/bin/env python3
import os
import argparse

parser = argparse.ArgumentParser(
    description="Merge into one file all files with given suffix whose size is less than specified."
)
parser.add_argument("suffix", help="file suffix")
parser.add_argument("size", type=int, help="maximum file size in bytes")
parser.add_argument("directory", help="directory to search")
parser.add_argument("outfile", help="output file")

```

```

args = parser.parse_args()
suffix = args.suffix
size = args.size
directory = args.directory
outfile = args.outfile
with open(outfile, "w") as out:
    for root, dirs, files in os.walk(directory):
        for name in files:
            if name.endswith("." + suffix):
                path = os.path.join(root, name)
                filesize = os.path.getsize(path)
                if filesize < size:
                    print("adding", path)
                    with open(path, "r") as f:
                        out.write(f.read())

```

9. **Дневник отладки** должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. **Замечания автора** по существу работы: замечания отсутствуют.

11. **Выводы:** В ходе выполнения лабораторной работы были разработаны два скрипта (на Bash и Python) для рекурсивного слияния файлов по заданному суффиксу и максимальному размеру. Реализована фильтрация по размеру и расширению, диагностический вывод имён добавляемых файлов. Оба скрипта успешно протестированы в среде ОС Ubuntu. Работа позволила закрепить навыки рекурсивного обхода файловой системы, работы с аргументами командной строки и обработки файлов на интерпретируемых языках.

Подпись студента _____